INSTITUTO FEDERAL
Espírito Santo
Campus Serra

ÁRVORES BINÁRIAS E ANÁLISE DE COMPLEXIDADE

Myllena Furtado Toniato
Rodrigo Lyrio Rodrigues
Ronnald Willian

Relatório desenvolvido para a disciplina de Técnicas de Programação Avançada do curso de Bacharelado em Sistemas de Informação do Instituto Federal do Espírito Santo – Campus Serra, como requisito parcial para avaliação.
Professor: Professor Victorio Albani de Carvalho

SERRA, ES
2026

SUMÁRIO

1. INTRODUÇÃO E ATRIBUIÇÃO DOS TÓPICOS.....3
1.1 OBJETIVO DO TRABALHO.....3
1.2 ATRIBUIÇÃO DOS INTEGRANTES.....3
1.3 LINK DO REPOSITÓRIO GITHUB.....3
2. ANÁLISE MATEMÁTICA DE COMPLEXIDADE - BIBLIOTECA AUTORAL.....5
2.1 MÉTODO ADICIONAR().....5
2.2 MÉTODO PESQUISAR().....7
2.3 MÉTODO REMOVER().....9
2.4 MÉTODO QUANTIDADENOS().....12
3. ANÁLISE EMPÍRICA DE COMPLEXIDADE DA ÁRVORE BINÁRIA COMUM (BST) DA BIBLIOTECA AUTORAL.....13
3.1. IMPLEMENTAÇÃO.....13
3.1.1 GERAÇÃO DOS ARQUIVOS DE ENTRADA.....13
3.1.2 MEDIÇÃO DOS TEMPOS DE EXECUÇÃO.....15
3.2 ANÁLISE DE COMPLEXIDADE.....17
3.2.1 LEITURA DOS ARQUIVOS.....17
3.2.2 PESQUISA PELO ÚLTIMO ELEMENTO.....18
3.2.3 PESQUISA POR ELEMENTO INEXISTENTE.....19
3.2.4 REMOÇÃO DO ÚLTIMO ELEMENTO.....20
3.2.5 QUANTIDADE DE NÓS.....21
4. AVALIAÇÃO DO DESEMPENHO DAS ÁRVORES AVL COMPARANDO COM AS ÁRVORES BINÁRIAS COMUNS.....22
4.1 CARACTERÍSTICAS DA ÁRVORE AVL.....22
4.2 COMPARAÇÃO DAS ANÁLISES DE COMPLEXIDADE.....26
4.2.1 DIFERENÇA NA LEITURA DOS ARQUIVOS.....26
4.2.2 DIFERENÇA NA PESQUISA PELO ÚLTIMO ELEMENTO.....27
4.2.3 DIFERENÇA NA PESQUISA POR ELEMENTO INEXISTENTE.....28
4.2.4 DIFERENÇA NA REMOÇÃO DO ÚLTIMO ELEMENTO.....29
4.2.5 COMPARAÇÃO DA QUANTIDADE DE NÓS.....30
5. ESTRUTURAS DE ÁRVORES NA BIBLIOTECA PADRÃO DO JAVA.....32
6. CONCLUSÃO.....34

1. INTRODUÇÃO E ATRIBUIÇÃO DOS TÓPICOS

1.1 OBJETIVO DO TRABALHO

O presente relatório tem como propósito realizar uma análise comparativa e detalhada da complexidade computacional das operações fundamentais de inserção, busca e remoção em estruturas de dados do tipo Árvore Binária de Busca (BST) e Árvore AVL. O atual estudo abrange tanto a fundamentação teórica quanto a validação empírica da eficiência algorítmica, observando como a topologia da árvore influencia o desempenho do sistema sob diferentes volumes de dados.

1.2 ATRIBUIÇÃO DOS INTEGRANTES

A divisão de responsabilidades foi estruturada de modo a garantir a integridade técnica do projeto, desde o gerenciamento de infraestrutura até a implementação de módulos de teste e análise de performance:

- **Myllena Furtado Toniato**: Encarregada do desenvolvimento do aplicativo de referência utilizando a biblioteca padrão do Java (AppJavaStandard). Além disso, implementou os módulos de automação para geração de dados, criando scripts responsáveis pela síntese de arquivos com registros balanceados e ordenados.

- **Rodrigo Lyrio Rodrigues**: Responsável pelo gerenciamento e estruturação do repositório. Atuou no desenvolvimento da biblioteca ArvoreAVL, na etapa de pré-processamento de dados (geração dos arquivos em formato .txt com registros de séries) e na execução de testes necessários para a correlação entre os resultados práticos e a teoria da complexidade.

- **Ronnald Willian**: Atuou no desenvolvimento da estrutura base do projeto. Implementou a classe NoArvore.java, adicionando o campo altura necessário para o cálculo do fator de balanceamento da Árvore AVL. Na classe ArvoreBinaria, configurou o construtor com o Comparator, e implementou os

métodos adicionar, pesquisar, remover, altura, caminharEmNível e caminharEmOrdem.

1.3 LINK DO REPOSITÓRIO GITHUB

Este trabalho está disponível em um repositório público na plataforma GitHub, cujo endereço de acesso está disponível a seguir: https://github.com/rodrigolyrio/complexidade-de-arvores-java.

2. ANÁLISE MATEMÁTICA DE COMPLEXIDADE - BIBLIOTECA AUTORAL

Nesta seção será feito o desenvolvimento matemático acerca das análises complexidades dos métodos adicionar, pesquisar, remover e quantidadeNos das árvores binárias e árvores AVL. Para todos os métodos, o relatório considerou o pior dos casos em árvores perfeitamente balanceadas e em árvores completamente degeneradas, respectivamente.

2.1 MÉTODO ADICIONAR()

Os métodos “adicionar” e “adicionarRecursivo” presentes na classe “ArvoreBinaria” são apresentados na imagem a seguir.

```
32      public void adicionar(T novoValor) {
33          int[] inserido = {0};
34          raiz = adicionarRecursivo(raiz, novoValor, inserido);
35          totalNos += inserido[0];
36      }
37
38      private NoArvore<T> adicionarRecursivo(NoArvore<T> no, T valor, int[] inserido) {
39          if (no == null) {
40              inserido[0] = 1;
41              return new NoArvore<>(valor);
42          }
43          int cmp = comparador.compare(valor, no.valor);
44          if (cmp < 0) {
45              no.esquerda = adicionarRecursivo(no.esquerda, valor, inserido);
46          } else if (cmp > 0) {
47              no.direita = adicionarRecursivo(no.direita, valor, inserido);
48          }
49
50          return no;
51      }
```

A organização do método “adicionar” se deu como apresentado abaixo:

- Linha 32: Apresenta um método público e sem retorno chamado “adicionar”, que recebe um valor novo.
- Linha 33: Cria um array de inteiros chamado “inserido” que recebe somente o valor 0.
- Linha 34: A variável “raiz” recebe o método “AdicionarRecursivo” que inicializa a busca pela raiz, descendo pelos nós até que o local ideal seja encontrado e o novo nó inserido e retornado. A explicação mais detalhada desse método recebido é efetuada no tópico de sua respectiva linha.
- Linha 35: O total de nós é incrementado em zero ou um, a depender se o valor existir ou não na árvore.

Já o desenvolvimento do método auxiliar “adicionarRecursivo” construído para o método principal “adicionar” se introduz da forma abaixo:

- Linha 38: Conta com a inicialização do método “adicionarRecursivo” do tipo genérico de NoArvore passando um nó do tipo NoArvore genérico, um valor também genérico e o array de inteiro nomeado “inserido”.

- Linha 39: Condicional para checar se o nó é nulo.
- Linha 40: Caso a condicional seja atendida, o array “inserido” recebe o valor 1 na posição 0.
- Linha 41: O programa retorna um novo nó criado com o valor passado.
- Linha 43: O inteiro “cmp” recebe um comparador que compara o valor passado com o nó da árvore a ser examinado.
- Linha 44: A condicional é iniciada com base no resultado da comparação dada por “cmp”. Se ele for menor do que zero, indica que o valor é menor do que o do nó analisado e a condicional é atendida.
- Linha 45: O programa então realiza uma chamada recursiva para o nó esquerdo da árvore, dada pelo “no.esquerdo”. Enquanto o valor for menor do que o valor do nó atual, o processo será efetuado, atualizado e salvo em “no.esquerda”.
- Linha 46: Se a comparação (“cmp”) não for menor, mas maior do que zero, significa que o valor a ser inserido é maior do que o do nó atual, então o programa precisa percorrer o filho à direita do nó da árvore.
- Linha 47: Seguindo a mesma lógica da linha 45, agora o programa chama recursivamente o lado direito e atualiza o “no.direita” com o retorno da função.
- Linha 50: O método possui como retorno o próprio nó para que após as modificações nas subárvores os pais continuem apontando para os filhos corretamente.

Nesse método recursivo, não há o caso em que o comparador é zero, pois caso isso aconteça, não há a necessidade de que o novo valor seja inserido, já que ele já está em um dos nós da árvore.

Com a explicação do método, temos então a análise da ordem de complexidade dele na árvore perfeitamente balanceada e na árvore completamente degenerada.

No primeiro caso, a partir da decisão tomada por qual subárvore seguir, sendo ela a da direita ou a da esquerda, o caminho percorrido é sempre encurtado pela metade, apontando a complexidade como O(log n). Dessa forma, o

adicionarRecursivo percorre da raiz até a folha é proporcional à altura da árvore, já que ela é minimizada em relação ao número de nós.

Já no caso da árvore completamente degenerada, onde cada nó tem apenas um filho, a altura da árvore é igual ao número de nós, sendo preciso no pior dos casos percorrer todos os nós para adicionar o novo elemento. Portanto, a complexidade nesse caso é O(n).

2.2 MÉTODO PESQUISAR()

O método pesquisar também foi construído com auxílio de outro método, o “pesquisarRecursivo”, como mostrado a seguir:

```
57          public T pesquisar(T valor) {
58              return pesquisarRecursivo(raiz, valor);
59          }
60
61          private T pesquisarRecursivo(NoArvore<T> no, T valor) {
62              if (no == null) return null;
63              int cmp = comparador.compare(valor, no.valor);
64              if (cmp == 0) return no.valor;
65              if (cmp < 0) return pesquisarRecursivo(no.esquerda, valor);
66              return pesquisarRecursivo(no.direita, valor);
67          }
```

- Linha 57: Definição do método público “pesquisar” que recebe um valor T.
- Linha 58: Retorna o método “pesquisarRecursivo” que recebe o valor da raiz e o valor a ser pesquisado.

A lógica do método recursivo é:

- Linha 61: Método privado “pesquisarRecursivo” passando o nó da árvore e o valor, ambos genéricos.

- Linha 62: Condicional para checar se o nó é nulo, retornando null caso a condição seja atendida.
- Linha 63: A variável de valor inteiro “cmp” recebe a comparação do comparador com o valor em questão e o valor do nó a ser analisado.
- Linha 64: Se a variável “cmp” for igual a zero, o programa retorna o valor do nó, pois o valor existe na árvore e foi encontrado.
- Linha 65: Se for menor do que zero, retorna o próprio método de pesquisa recursiva, só que com o nó à esquerda do nó atual e o valor a ser buscado.
- Linha 66: Caso nenhuma das condições forem atendidas significa que o “cmp” é maior do que zero, ou seja, o valor é maior do que a raiz e o programa deve retornar o método recursivo de pesquisa com o valor do nó à direita do nó atual.

Seguindo a mesma lógica do método “adicionar”, o método de pesquisa de elementos mantém a ordem de complexidade da árvore perfeitamente balanceada como O(log n) e da árvore completamente degenerada como O(n), pois o programa continua decidindo entre esquerda e direita a cada nível que é analisado.

2.3 MÉTODO REMOVER()

Abaixo está o método de remoção de elementos da árvore, também utilizando-se da recursão como ferramenta para atingir seu objetivo.

```
76      public boolean remover(T valor) {
77          int[] removido = {0};
78          raiz = removerRecursivo(raiz, valor, removido);
79          if (removido[0] == 1) {
80              totalNos--;
81              return true;
82          }
83          return false;
84      }
85
86      private NoArvore<T> removerRecursivo(NoArvore<T> no, T valor, int[] removido) {
87          if (no == null) return null;
88
89          int cmp = comparador.compare(valor, no.valor);
90
91          if (cmp < 0) {
92              no.esquerda = removerRecursivo(no.esquerda, valor, removido);
93          } else if (cmp > 0) {
94              no.direita = removerRecursivo(no.direita, valor, removido);
95          } else {
96              // Nó encontrado
97              removido[0] = 1;
98
99              // Caso 1 e 2: zero ou um filho
100             if (no.esquerda == null) return no.direita;
101             if (no.direita == null) return no.esquerda;
102
103             // Caso 3: dois filhos. Substitui pelo sucessor em-ordem
104             NoArvore<T> sucessor = menorNo(no.direita);
105             no.valor = sucessor.valor;
106             // Remove o sucessor da subárvore direita (não marca como removido novamente)
107             no.direita = removerRecursivo(no.direita, sucessor.valor, new int[]{0});
108         }
109         return no;
110     }
```

Para a remoção do nó da árvore, primeiramente é necessário buscá-lo para constatar se ele existe e foi removido ou se não existe. O trecho da linha 76 á 97 trata dessa remoção.

- Linha 76: Construção do método público “remover” que retorna um valor booleano indicando a remoção ou não do elemento ao receber o valor a ser buscado.
- Linha 77: Array de inteiro nomeado “removido” que recebe um elemento – o zero.
- Linha 78: A variável raiz recebe o método “removerRecursivo” com a raiz, o valor e o array “removido”.

- Linha 79: Se a posição 0 do array for 1, o elemento foi encontrado e removido.
- Linha 80: O total de nós é decrementado.
- Linha 81: O programa retorna true para indicar que o objetivo do método foi concluído.
- Linha 83: Caso a condição anterior não seja atendida, o código retorna falso, ou seja, o elemento não foi encontrado.

A estrutura do método “removerRecursivo” segue a lógica a seguir:

- Linha 86: Início do método privado e recursivo de remoção do tipo NoArvore, do qual recebe o nó do tipo genérico NoArvore, o valor de interesse e o array “removido”.
- Linha 87: Condicional para verificar se o nó é nulo e retornar nulo caso seja.
- Linha 89: Acontece o armazenamento na variável inteira “cmp” do resultado da comparação entre o valor a ser removido e o valor do nó atual. A depender do resultado do “cmp”, o programa analisará o lado esquerdo ou o direito da árvore.
- Linha 91: O condicional é aplicado se o valor buscado é menor que o valor do nó atual.
- Linha 92: Caso seja menor, o valor está à esquerda, portanto o resultado da chamada recursiva é atualizado no ponteiro “no.esquerda”.
- Linha 93: Caso contrário, o valor é maior do que o nó atual e estará do lado direito.
- Linha 94: O ponteiro “no.direita” atualiza com o resultado da chamada recursiva para seu filho direito.
- Linha 97: Além das duas situações (maior ou menor que a raiz), há a hipótese do valor ser igual à ela, o que indica que ele foi encontrado. Nesse caso, a posição 0 do array “removido” é atualizada para 1, sinalizando para a linha 79 que a exclusão foi executada com sucesso.

Assim que o elemento for removido, aquele nó precisa ser ajustado para que outro valor o substitua. A linha 100 até a linha 107 resolve essa situação em que três casos são possíveis: o nó possui 0, 1 ou 2 filhos.

- Linha 100: Trata o caso de que o nó analisado não possui filho à esquerda (“no.esquerda” é nulo), ou seja, o seu sucessor será seu filho à direita (“no.direita”).
- Linha 101: A próxima possibilidade é que o nó não possui filho à direita (“no.direita” é nulo), fazendo com que o programa retorne seu filho à esquerda (“no.esquerda”).

Em situações em que o nó possui dois filhos (linha 104 adiante), para removê-lo é preciso buscar seu sucessor em-ordem (o menor valor dá subárvore direita).

- Linha 104: Chama o método privado “menorNo” do tipo genérico “NoArvore” para encontrar o nó sucessor em-ordem do nó à direita da raiz.

Para esse trecho, outro método foi utilizado:

```
113        private NoArvore<T> menorNo(NoArvore<T> no) {
114            while (no.esquerda != null) {
115                no = no.esquerda;
116            }
117            return no;
118        }
```

- Linha 113: Constroi o método usado na linha 104.
- Linha 114: Apresenta um looping enquanto o nó esquerdo não for nulo.
- Linha 115: Com a condição atendida, o programa busca o nó mais à esquerda, até que o nó não tenha mais filhos à esquerda.
- Linha 117: Por fim, o método retorna o nó de menor valor (o mais à esquerda) daquela árvore.

A ordem de complexidade tanto da árvore perfeitamente balanceada quanto da árvore completamente degenerada é igual às ordens anteriores (O(log n) e O(n) respectivamente), já que a árvore degenerada é uma linha reta para a direita ou para a esquerda em que é preciso percorrer todos os nós no pior dos casos e a árvore balanceada o caminho percorrido é dividido ao meio a cada nível percorrido.

2.4 MÉTODO QUANTIDADENOS()

No método público e inteiro “quantidadeNos”, que não recebe parâmetros, há o retorno da variável contadora de nós (totalNos).

```
124         public int quantidadeNos() {
125             return totalNos;
126         }
```

Da mesma forma que na lista encadeada, nesse método a variável é atualizada durante cada remoção ou incremento de itens novos à lista, portanto a estrutura da árvore não interfere no desempenho do método e a complexidade da operação continua sendo O(1) para o pior dos casos dos dois tipos de árvores observadas.

3. ANÁLISE EMPÍRICA DE COMPLEXIDADE DA ÁRVORE BINÁRIA COMUM (BST) DA BIBLIOTECA AUTORAL

Nesta seção, é apresentada a metodologia aplicada para a validação prática dos algoritmos de árvores binárias implementados. O objetivo desta análise é observar o comportamento real dessa estrutura sob diferentes volumes de carga e topologias, permitindo uma comparação direta com os modelos teóricos de complexidade O(n) e O(log n).

3.1. IMPLEMENTAÇÃO

A metodologia de testes foi estruturada para isolar as variáveis de desempenho, garantindo que as medições reflitam exclusivamente o esforço computacional das operações de manipulação de dados.

3.1.1 GERAÇÃO DOS ARQUIVOS DE ENTRADA

Para a realização dos experimentos, foram sintetizados arquivos contendo 100 mil, 200 mil e 400 mil registros de séries, mantendo a métrica de volumetria utilizada em trabalhos anteriores. Seguindo as diretrizes da especificação, geraram-se duas categorias de arquivos para cada volume, visando induzir diferentes topologias na árvore:

- **Arquivos para Árvores Ordenadas**: Utilizou-se o **GeradorArquivosOrdenados.java** para criar chaves de busca em ordem crescente. Ao serem inseridos na árvore binária comum (BST), esses registros provocam a degeneração da estrutura em uma lista encadeada, forçando o pior caso de complexidade, O(n). Aqui, o “segredo” é o método recursivo que encontra sempre o elemento do meio.

```
84        // Mantém a lógica de IDs balanceados para evitar árvores degeneradas
85        private static void gerarIdentificadoresBalanceados(ArrayList<Integer> ids, int inicio, int fim) {
86            if (inicio > fim) return;
87            int meio = (inicio + fim) / 2;
88            ids.add(meio);
89            gerarIdentificadoresBalanceados(ids, inicio, meio - 1);
90            gerarIdentificadoresBalanceados(ids, meio + 1, fim);
91        }
```

Explicação Linha a Linha:

- 86: **if (inicio > fim) return;**: É a condição de parada. Se o intervalo acabar, a função encerra.
- 87: **int meio = (inicio + fim) / 2**;: Calcula o ponto central do intervalo atual.
- 88: **ids.add(meio)**;: Adiciona esse valor central na lista. Como ele entra primeiro, ele se torna a raiz (ou pai) daquela parte da árvore.
- 89: **gerarIdentificadoresBalanceados(..., inicio, meio - 1)**;: Chama a si mesma para resolver a metade da esquerda.
- 90: **gerarIdentificadoresBalanceados(..., meio + 1, fim)**;: Chama a si mesma para resolver a metade da direita.

- **Arquivos para Árvores Balanceadas**: Através do **GeradorArquivosBalanceados.java**, os dados foram dispostos de modo que cada inserção divida o intervalo de chaves ao meio. Essa técnica assegura uma altura mínima para a estrutura, permitindo testar o desempenho ideal de O(log n).

```
63        // Converte para lista e ordena alfabeticamente [Essencial para o pior caso]
64        ArrayList<String> nomesOrdenados = new ArrayList<>(nomesUnicos);
65        Collections.sort(nomesOrdenados);
66
67        System.out.println("Gravando arquivo ordenado...");
68        try (BufferedWriter writer = new BufferedWriter(new FileWriter(NOME_ARQUIVO, StandardCharsets.UTF_8))) {
69            writer.write(NUM_REGISTROS + "\n");
70
71            for (String nome : nomesOrdenados) {
72                int anoLancamento = 1950 + random.nextInt(75);
73                String pais = PAISES[random.nextInt(PAISES.length)];
74
75                // Formato: nome;ano;pais
76                writer.write(nome + ";" + anoLancamento + ";" + pais + "\n");
77            }
```

Explicação Linha a Linha:

- 64: **ArrayList<String> nomesOrdenados = new ArrayList<>(nomesUnicos)**;: Pega todos os nomes aleatórios gerados e joga em uma lista para facilitar a manipulação.
- 65: **Collections.sort(nomesOrdenados)**;: Organiza todos os nomes em ordem alfabética estrita (A-Z).
- 71: **for (String nome : nomesOrdenados)**: Inicia um laço que percorre essa lista já organizada.
- 76: **writer.write(...)**: Grava os dados no arquivo exatamente nessa sequência.

3.1.2 MEDIÇÃO DOS TEMPOS DE EXECUÇÃO

A coleta de dados de performance foi realizada através da classe utilitária MedidorTempo.java, que utiliza o método nativo System.nanoTime() para garantir precisão na ordem de nanossegundos. A escolha por nanossegundos é justificada pela alta velocidade das operações em árvores balanceadas, que frequentemente ocorrem em frações de milissegundo.

A lógica de medição foi encapsulada em métodos, permitindo cronometrar isoladamente as operações de adição, pesquisa e remoção sem a interferência do tempo de processamento do menu ou entrada de dados do usuário. Os valores foram posteriormente convertidos para milissegundos (ms) para facilitar a leitura e análise nas tabelas comparativas.

```java
5  public class MedidorTempo {
6
7      public static long medir(Runnable acao) {
8          long inicio = System.nanoTime(); // Marca o tempo inicial em nanossegundos
9          acao.run(); // Executa a ação passada como parâmetro
10         long fim = System.nanoTime(); // Marca o tempo final após execução
11         return fim - inicio;
12     }
13
14     public static <T> ResultadoMedicao<T> medirComRetorno(Supplier<T> acao) {
15         long inicio = System.nanoTime(); // Marca o tempo inicial
16         T resultado = acao.get(); // Executa a ação e armazena o resultado retornado
17         long fim = System.nanoTime(); // Marca o tempo final
18         return new ResultadoMedicao<T>(resultado, fim - inicio);
19     }
20
21     public static void imprimirTempo(String descricao, long tempoNano) {
22         System.out.println(descricao + ": " + (tempoNano / 1_000_000.0) + " ms\n");
23     }
24 }
```

Explicação Linha a Linha:

- 7: **public static long medir(Runnable acao)**: Cria uma "caixa" que recebe qualquer tarefa (como pesquisar ou inserir) para ser cronometrada isoladamente.
- 8: **long inicio = System.nanoTime()**;: Aciona o cronômetro do processador com precisão de nanossegundos antes da tarefa começar.
- 9: **acao.run()**;: Executa a operação real na árvore. Como isso acontece entre o início e o fim, o tempo de menus ou teclado não interfere na medição.
- 10: **long fim = System.nanoTime()**;: Trava o cronômetro assim que a operação termina.
- 17: **return fim - inicio;**: Faz a conta de subtração para saber exatamente quanto tempo a tarefa levou.
- 22: **(tempoNano / 1_000_000.0)**: Divide o valor bruto por 1 milhão. Isso transforma aqueles números gigantes de nanossegundos em milissegundos (ms), que são mais fáceis de ler nas tabelas.

3.2 ANÁLISE DE COMPLEXIDADE

Nesta subseção, interpretamos os dados obtidos nos testes de estresse, confrontando o desempenho prático da Árvore Binária de Busca (BST) com as predições matemáticas. O foco aqui é observar como a topologia da estrutura dita a velocidade das operações.

3.2.1 LEITURA DOS ARQUIVOS

Nesta etapa, analisamos o tempo necessário para carregar os registros do disco e inseri-los na Árvore Binária de Busca (BST). Abaixo, apresentamos os dados coletados nos testes de estresse para os três volumes definidos:

Volume	Tipo de Arquivo	Teste 01 (ms)	Teste 02 (ms)	Teste 03 (ms)	Média (ms)
100k	**Balanceado**	172,57	170,97	183,46	**175,67**
100k	**Ordenado**	127,59	118,33	123,05	**122,99**
200k	**Balanceado**	310,10	329,43	324,09	**321,21**
200k	**Ordenado**	201,81	214,93	196,75	**204,50**
400k	**Balanceado**	549,37	576,96	565,30	**563,88**
400k	**Ordenado**	357,47	363,71	335,38	**352,19**

Tabela 1: Tempos de Leitura e Inserção (BST)

Os dados confirmam um crescimento linear (O(n)) no tempo de carregamento. Quando o volume de dados quadruplica (de 100 mil para 400 mil), o tempo médio de inserção também aumenta em proporção similar.

Note que o arquivo Ordenado é carregado mais rápido que o Balanceado. Isso ocorre porque a árvore comum não perde tempo com rotações ou lógica de equilíbrio; ela apenas "empilha" os dados para a direita, criando uma estrutura degenerada (em forma de lista) que será extremamente lenta em buscas futuras.

3.2.2 PESQUISA PELO ÚLTIMO ELEMENTO

O "último elemento" refere-se ao registro que, pela lógica de inserção, encontra-se no nível mais profundo da estrutura.

Volume	Tipo de Arquivo	Teste 01 (ms)	Teste 02 (ms)	Teste 03 (ms)	Média (ms)
100k	Balanceado	0,0540	0,0560	0,0510	**0,0536**
100k	Ordenado	1,7397	1,8415	1,8285	**1,8032**
200k	Balanceado	0,0641	0,0713	0,0650	**0,0668**
200k	Ordenado	3,6339	3,4912	3,3986	**3,5079**
400k	Balanceado	0,0593	0,0633	0,0572	**0,0599**
400k	Ordenado	7,3995	7,3828	7,3238	**7,3687**

Tabela 2: Tempos de Pesquisa pelo Último Elemento (BST)

Os resultados revelam um contraste drástico entre as topologias. No arquivo Balanceado, o tempo permanece praticamente constante e extremamente baixo (médias abaixo de 0,07 ms), caracterizando a eficiência O(log n). No arquivo Ordenado, o tempo de busca dobra conforme o volume de dados dobra, confirmando a degeneração para a complexidade O(n).

Enquanto na árvore balanceada de 400 mil a busca leva apenas 0,05 ms, na árvore degenerada o tempo sobe para 7,36 ms. Isso prova que, em uma estrutura tipo "lista", o processador precisa realizar centenas de milhares de comparações a mais para alcançar o mesmo objetivo.

3.2.3 PESQUISA POR ELEMENTO INEXISTENTE

Esta análise visa observar o comportamento do algoritmo de busca quando o elemento alvo não faz parte da estrutura. Nestes casos, o algoritmo é forçado a percorrer um caminho completo da raiz até atingir uma extremidade nula (folha) para confirmar a ausência do registro.

Volume	Tipo de Arquivo	Teste 01 (ms)	Teste 02 (ms)	Teste 03 (ms)	Média (ms)
100k	Balanceado	0,0492	0,0439	0,0669	**0,0533**
100k	Ordenado	1,7645	1,7770	1,7698	**1,7704**
200k	Balanceado	0,0638	0,0689	0,0628	**0,0651**
200k	Ordenado	3,5284	3,5133	3,5532	**3,5316**
400k	Balanceado	0,0694	0,0683	0,0608	**0,0661**
400k	Ordenado	7,2144	7,3916	7,3323	**7,3127**

Tabela 3: Tempos de Pesquisa por Elemento Inexistente (BST)

Assim como na busca pelo último elemento, a pesquisa por um item inexistente em arquivos Ordenados demonstra um crescimento linear de tempo. Ao dobrar o número de registros, o tempo de resposta praticamente dobra (de ~3,53 ms para ~7,31 ms), validando a complexidade O(n) da estrutura degenerada.

Na estrutura Balanceada, a busca falha rapidamente independentemente do tamanho do arquivo, mantendo médias estáveis próximas a 0,06 ms. Isso ocorre porque, mesmo na falha, o algoritmo percorre apenas a altura da árvore, que em uma topologia balanceada é log n.

3.2.4 REMOÇÃO DO ÚLTIMO ELEMENTO

A operação de remoção testa a capacidade da árvore de localizar um nó específico e ajustar seus ponteiros internos para manter a integridade da estrutura. Neste teste, removemos o elemento inserido por último, o que exige que o algoritmo navegue até a profundidade máxima da árvore.

Volume	Tipo de Arquivo	Teste 01 (ms)	Teste 02 (ms)	Teste 03 (ms)	Média (ms)
100k	Balanceado	0,0512	0,0589	0,0531	**0,0544**
100k	Ordenado	1,8245	1,7912	1,8560	**1,8239**
200k	Balanceado	0,0610	0,0722	0,0645	**0,0659**
200k	Ordenado	3,6511	3,5980	3,6122	**3,6204**
400k	Balanceado	0,0685	0,0711	0,0632	**0,0676**
400k	Ordenado	7,4210	7,4556	7,3890	**7,4218**

Tabela 4: Tempos de Remoção do Último Elemento (BST)

Assim como observado nas pesquisas, a remoção em arquivos Ordenados demonstra um custo temporal muito superior. Isso ocorre porque, antes de deletar o nó, o sistema precisa realizar uma busca linear até o fim da "lista", resultando em complexidade O(n).

Na topologia Balanceada, o tempo de remoção é quase instantâneo e se mantém estável mesmo com o aumento massivo de dados. O algoritmo atinge o nó alvo em poucos passos (altura log n), provando que uma árvore bem estruturada facilita tanto a entrada quanto a saída de informações.

3.2.5 QUANTIDADE DE NÓS

Esta operação tem como objetivo retornar o número total de elementos armazenados na árvore. Graças a nossa biblioteca, que mantém um contador atualizado em tempo real, a operação atinge o nível máximo de eficiência teórica.

Volume	Tipo de Arquivo	Teste 01 (ms)	Teste 02 (ms)	Teste 03 (ms)	Média (ms)
100k	Balanceado	0.0116	0.0141	0.0125	**0.0127**
100k	Ordenado	0.0280	0.0125	0.0131	**0.0179**
200k	Balanceado	0.0141	0.0103	0.0104	**0.0116**
200k	Ordenado	0.0117	0.0208	0.0108	**0.0144**
400k	Balanceado	0.0092	0.0106	0.0137	**0.0112**
400k	Ordenado	0.0098	0.0100	0.0114	**0.0104**

Tabela 5: Tempos de Execução do Método quantidadeNos() (BST)

Repare que, mesmo quando o volume de dados sobe de 100 mil para 400 mil registros, a média de tempo não aumenta; ela até diminui levemente ou oscila na mesma faixa. Isso prova que o algoritmo não precisa "varrer" a árvore, confirmando a complexidade O(1).

Não importa se a árvore está balanceada ou se virou uma "lista" (ordenada); o tempo de resposta é o mesmo. Isso acontece porque não há navegação entre os nós para realizar a contagem.

4. AVALIAÇÃO DO DESEMPENHO DAS ÁRVORES AVL COMPARANDO COM AS ÁRVORES BINÁRIAS COMUNS

Enquanto a árvore binária comum é "refém" da ordem de entrada dos dados, a AVL utiliza uma inteligência própria para nunca deixar o desempenho cair.

O objetivo aqui é avaliar os tempos que coletamos na seção 3 com os novos resultados da AVL, utilizando exatamente os mesmos arquivos de 100 mil, 200 mil e 400 mil registros.

4.1 CARACTERÍSTICAS DA ÁRVORE AVL

A Árvore AVL surge como uma solução direta para o problema da degeneração observado na seção anterior. Enquanto uma BST comum aceita crescer de forma desordenada, a AVL é uma árvore binária de busca auto-balanceada, o que significa que ela se reorganiza sozinha a cada nova inserção ou remoção.

As principais características técnicas implementadas em nossa biblioteca são:

- **Fator de Equilíbrio**: Para cada nó, calculamos a diferença entre a altura da subárvore esquerda e a da direita. A regra é que esse valor deve estar sempre entre -1, 0 e 1.
- **Custo de Manutenção**: Diferente da árvore comum, a AVL exige um esforço extra de processamento durante a inserção, pois precisa atualizar alturas e verificar o equilíbrio de todos os nós ancestrais.
- **Mecanismo de Rotações**: Quando o Fator de Equilíbrio viola a regra (Fator de Equilíbrio > 1 ou Fator de Equilíbrio < -1), o código dispara ‘manobras’ chamadas rotações (simples ou duplas) para reduzir a altura do galho mais pesado.

- **Garantia de Performance**: Graças a esse auto-ajuste, a árvore nunca se torna uma lista. Isso garante que, mesmo no pior cenário (dados ordenados), a complexidade de busca seja sempre O(log n).

O diferencial da classe ArvoreAVL.java em relação à binária comum está nos métodos de rebalanceamento. Abaixo, detalhamos como a árvore "percebe" que está torta e como ela se conserta.

```
50      private NoArvore<T> rotacaoDireita(NoArvore<T> y) {
51          NoArvore<T> x = y.esquerda; // x é nó filho esquerdo de y.
52          NoArvore<T> T2 = x.direita; // T2 é nó filho direito de x.
53
54          // ROTACIONA PARA A DIREITA...
55          x.direita = y;
56          y.esquerda = T2;
57          // Ao realizar essa rotação, x se torna nó pai do nó y e do nó T2.
58
59          atualizarAltura(y);
60          atualizarAltura(x);
61
62          return x; // Retorna a nova raiz (ou o novo nó pai)
63      }
```

Explicação Linha a Linha:

- 51: **NoArvore<T> x = y.esquerda;**: Identifica o filho à esquerda que subirá para a posição de raiz local.
- 52: **NoArvore<T> T2 = x.direita;**: Reserva a subárvore à direita do filho (x) para que ela seja realocada sem perda de dados.
- 55: **x.direita = y;**: Realiza o giro: o nó que estava em desequilíbrio (y) "desce" e se torna o novo filho à direita de x.
- 56: **y.esquerda = T2;**: A subárvore reservada anteriormente é conectada no espaço vago à esquerda de y.
- 59-60: **atualizarAltura(y); atualizarAltura(x)**;: Recalcula as alturas dos nós afetados para que o próximo cálculo de equilíbrio seja preciso.

```
69      private NoArvore<T> rotacaoEsquerda(NoArvore<T> x) {
70          NoArvore<T> y = x.direita;
71          NoArvore<T> T2 = y.esquerda;
72
73          y.esquerda = x;
74          x.direita = T2;
75
76          atualizarAltura(x);
77          atualizarAltura(y);
78
79          return y;
80      }
```

Explicação Linha a Linha:

- 70: **NoArvore<T> y = x.direita;**: Seleciona o filho à direita que se tornará o novo pai local.
- 71: **NoArvore<T> T2 = y.esquerda;**: Guarda a subárvore à esquerda de y para evitar que ela seja sobrescrita.
- 73: **y.esquerda = x;**: Faz o nó x descer, tornando-se filho à esquerda de y.
- 74: **x.direita = T2;**: Conecta a subárvore guardada no lugar que ficou livre à direita de x.

```
181            atualizarAltura(no);
182            int balanceamento = getFatorBalanceamento(no);
183
184            if (balanceamento > 1) {
185                if (getFatorBalanceamento(no.esquerda) >= 0) {
186                    return rotacaoDireita(no);
187                } else {
188                    no.esquerda = rotacaoEsquerda(no.esquerda);
189                    return rotacaoDireita(no);
190                }
191            }
192
193            if (balanceamento < -1) {
194                if (getFatorBalanceamento(no.direita) <= 0) {
195                    return rotacaoEsquerda(no);
196                } else {
197                    no.direita = rotacaoDireita(no.direita);
198                    return rotacaoEsquerda(no);
199                }
200            }
```

Explicação Linha a Linha:

- 181: **atualizarAltura(no)**;: Antes de qualquer coisa, o código recalcula a altura do nó atual, pois a remoção de um filho pode ter mudado o tamanho do galho.
- 182: **int balanceamento = getFatorBalanceamento(no)**;: Aqui o código "pesa" a árvore, subtraindo a altura da esquerda pela da direita para ver se ela saiu do eixo.
- 184: **if (balanceamento > 1)**: Se o resultado for maior que 1, o código detecta que o lado esquerdo está muito pesado e precisa de uma intervenção.
- 185: **if (getFatorBalanceamento(no.esquerda) < 0)**: O código olha para o filho da esquerda. Se ele estiver pendendo para o lado oposto (direita), temos um ziguezague. Para resolver, fizemos primeiro uma Rotação Esquerda no filho para alinhar a árvore antes da manobra principal.

- 189: **return rotacaoDireita(no)**;: Com tudo alinhado, o código executa a rotação final para a direita, trazendo o equilíbrio de volta e retornando o novo nó que ficará no topo.
- **193**: **if (balanceamento < -1)**: Esta é a lógica espelhada para quando o lado direito está mais pesado, garantindo que a árvore seja corrigida não importa para qual lado ela tombe.

4.2 COMPARAÇÃO DAS ANÁLISES DE COMPLEXIDADE

Nesta seção, validamos empiricamente a superioridade da Árvore AVL sobre a Árvore Binária Comum em cenários de estresse. Enquanto a BST comum é altamente dependente da ordem de entrada, a AVL mantém uma topologia otimizada que reflete diretamente nos tempos de execução.

4.2.1 DIFERENÇA NA LEITURA DOS ARQUIVOS

Nesta subseção, comparamos o tempo que o sistema leva para ler os registros do disco e inseri-los na estrutura. Esse processo é o "teste de fogo" para o custo de manutenção da AVL, já que ela precisa verificar o equilíbrio a cada novo nó.

Volume	Tipo de Arquivo	Média BST (ms)	Média AVL (ms)
100k	Balanceado	175,67	~172,45
100k	Ordenado	122,99	**~105,21**
200k	Balanceado	321,21	~313,43
200k	Ordenado	204,50	**~168,74**
400k	Balanceado	563,88	~585,98

400k	Ordenado	352,19	**~269,60**

Tabela 6: Comparativo de Inserção (BST vs. AVL)

Ao reparar nos arquivos balanceados de 400 mil, a AVL levou 585,98 ms, enquanto a BST foi um pouco mais rápida com 563,88 ms. Isso acontece porque a AVL gasta um tempinho a mais rodando o método rebalancear e atualizando alturas, mesmo que a árvore já esteja quase equilibrada. Em arquivos ordenados, a AVL foi muito mais rápida (269,60 ms contra 352,19 ms da BST para 400 mil registros).

Na BST comum, quando os dados estão ordenados, a árvore vira uma "lista" gigante. Para inserir o elemento 400.000, o computador tem que percorrer os 399.999 anteriores. Na AVL, as rotações mantêm a árvore baixa, então o caminho até o lugar de inserção é sempre curto, compensando de longe o esforço de rotacionar.

4.2.2 DIFERENÇA NA PESQUISA PELO ÚLTIMO ELEMENTO

O "último elemento" é aquele que está no nível mais profundo da árvore, exigindo o maior número de comparações para ser localizado.

Volume	Tipo de Arquivo	Média BST (ms)	Média AVL (ms)
100k	Balanceado	0,0536	~0,0690
100k	Ordenado	1,8032	**~0,0588**
200k	Balanceado	0,0668	~0,0744
200k	Ordenado	3,5079	**~0,0574**
400k	Balanceado	0,0599	~0,0742

400k	Ordenado	**7,3687**	**~0,0645**

Tabela 7: Comparativo de Busca pelo Último Elemento (BST vs. AVL)

Enquanto a BST Comum entrou em colapso com os dados ordenados, saltando de 1,80 ms para 7,36 ms conforme o volume crescia, a AVL manteve o tempo praticamente constante e baixíssimo (cerca de 0,06 ms).

Para o volume de 400 mil registros ordenados, a Árvore AVL foi aproximadamente 114 vezes mais rápida que a BST comum. Isso prova que as rotações que fizemos lá no adicionarRecursivo garantiram que a altura da árvore nunca passasse do limite logarítmico.

Na BST comum, a busca pelo último elemento em dados ordenados virou uma operação O(n) (linear), onde o processador precisou visitar quase todos os nós como se fosse uma lista. Já na AVL, a operação permaneceu fiel ao O(log n), encontrando o alvo em pouquíssimos saltos.

4.2.3 DIFERENÇA NA PESQUISA POR ELEMENTO INEXISTENTE

Nesta análise, testamos como o sistema reage quando o registro buscado não faz parte da estrutura. Esse teste é crucial porque força o algoritmo a percorrer um caminho completo da raiz até atingir uma extremidade nula (folha) para confirmar a ausência do dado.

Volume	Tipo de Arquivo	Média BST (ms)	Média AVL (ms)
100k	Balanceado	0,0533	~0,0476
100k	Ordenado	1,7704	**~0,0471**
200k	Balanceado	0,0651	~0,0563

200k	Ordenado	3,5316	**~0,0516**
400k	Balanceado	0,0661	~0,0579
400k	Ordenado	**7,3127**	**~0,0524**

Tabela 8: Pesquisa por Elemento Inexistente (BST vs. AVL)

Repare que a Árvore AVL mantém tempos baixíssimos e quase idênticos (~0,05 ms) tanto para arquivos balanceados quanto para ordenados. Isso prova que, graças ao auto-balanceamento, o "pior caminho" que o algoritmo precisa percorrer é sempre curto, respeitando a complexidade O(log n).

Na BST Comum, a falta de equilíbrio cobra seu preço. No arquivo ordenado de 400 mil registros, o tempo chegou a 7,31 ms. Como a árvore se tornou uma lista, o processador teve que checar quase todos os 400 mil nós antes de dizer que o dado realmente não está presente.

4.2.4 DIFERENÇA NA REMOÇÃO DO ÚLTIMO ELEMENTO

Esta operação testa a capacidade da árvore de localizar um nó específico — no caso, a folha mais profunda — e ajustar seus ponteiros internos para manter a integridade da estrutura. Na AVL, esse processo é ainda mais complexo, pois além de remover, o código precisa "rebalancear" o caminho de volta até a raiz.

Volume	Tipo de Arquivo	Média BST (ms)	Média AVL (ms)
100k	Balanceado	0,0544	**~0,0628**
100k	Ordenado	1,8239	**~0,0640**
200k	Balanceado	0,0659	**~0,0708**

200k	Ordenado	3,6204	**~0,0777**
400k	Balanceado	0,0676	**~0,0496**
400k	Ordenado	**7,4218**	**~0,0619**

Tabela 9: Comparativo de Remoção do Último Elemento (BST vs. AVL)

Em arquivos ordenados de 400 mil registros, enquanto a BST Comum demora 7,42 ms para encontrar e remover a folha, a AVL faz isso em apenas 0,06 ms. Isso significa que a AVL é cerca de 120 vezes mais rápida nesse cenário de estresse.

Em arquivos balanceados, os tempos são muito próximos. A AVL gasta alguns microssegundos a mais rodando o método rotacaoDireita ou rotacaoEsquerda para garantir que a árvore não entorte após a saída do nó.

Na BST comum com dados ordenados, a remoção sofre com o O(n) porque o algoritmo precisa caminhar por toda a lista até o final. Na AVL, a altura é sempre O(log n), então a busca pelo nó a ser removido é feita de maneira extremamente rápida.

4.2.5 COMPARAÇÃO DA QUANTIDADE DE NÓS

O propósito desta operação consiste em retornar a cardinalidade total da estrutura de dados. Em virtude do design otimizado adotado em ambas as implementações (BST e AVL), os resultados experimentais demonstram uma convergência de performance, evidenciando paridade nos tempos de execução independentemente da topologia ou volumetria utilizada.

Volume	Tipo de Arquivo	Média BST (ms)	Média AVL (ms)
100k	Balanceado	0.0127	~0.0048

100k	Ordenado	0.0179	~0.0064
200k	Balanceado	0.0116	~0.0057
200k	Ordenado	0.0144	~0.0055
400k	Balanceado	0.0112	~0.0061
400k	Ordenado	0.0104	~0.0057

Tabela 10: Comparativo de Consulta de Tamanho (BST vs. AVL)

A análise dos dados experimentais revela que, independentemente da volumetria dos arquivos de entrada (100 mil a 400 mil registros), o tempo de resposta da operação permaneceu na ordem de grandeza de centésimos de milissegundo. Este comportamento evidencia que o tempo de execução é independente do número de elementos (n) presentes na estrutura, o que corrobora a complexidade algorítmica constante (O(1)) estabelecida na fundamentação teórica.

Verificou-se, ainda, uma completa independência topológica nesta operação. O desempenho manteve-se uniforme tanto em árvores perfeitamente balanceadas quanto em estruturas totalmente degeneradas (ordenadas), uma vez que o algoritmo prescinde do percorrimento dos ramos para determinar a cardinalidade da árvore.

Esta eficiência computacional é atribuída à implementação do atributo protegido totalNos. Em vez de instanciar um caminhamento recursivo a cada requisição do usuário, a biblioteca gerencia um contador persistente que sofre incremento ou decremento imediato durante as operações de inserção e remoção, respectivamente. Consequentemente, a chamada do método limita-se a um acesso direto à memória para leitura do valor previamente computado.

5. ESTRUTURAS DE ÁRVORES NA BIBLIOTECA PADRÃO DO JAVA

Esta seção disserta acerca da estrutura de árvore binária disponibilizada pela biblioteca padrão de Java da qual foi aplicada na classe “AppJavaStandard” localizada na pasta “app” do GitHub.

O Java utiliza-se do TreeSet como estrutura de dados base. O TreeSet por sua vez utiliza uma árvore binária de busca auto balanceada, em que cada operação de inserção (add) e remoção (remove) de elementos a árvore é ajustada, garantindo a consistência de performance para as operações de busca, inserção e remoção.

Tanto a operação de inserção (add) quanto a de busca (floor) e remoção (remove) de um elemento novo faz com que após a busca para encontrar a posição correta do nó, o rebalanceamento da árvore precisa acontecer para manter a eficiência nas próximas operações, o que significa que a ordem de complexidade é sempre O(log n) no pior dos casos.

O método de busca é através do “floor”, entretanto existem outras formas de procurar um elemento de uma árvore pela estrutura padrão do Java, como o contains, first, last, lower, higher e ceiling. A decisão tomada foi a partir do tipo de retorno que o “floor” entrega – o elemento menor ou igual ao valor desejado – enquanto o método “contains”, por exemplo, é booleano e portanto retorna somente true ou false, sem apresentar o elemento em si. Os outros métodos são para achar um elemento localizado em uma parte específica da árvore, seja o primeiro ou o último elemento, por exemplo, enquanto o “floor” possibilita ao usuário a escolha de um nó específico. Apesar da decisão de um método em específico, todos os métodos – não só os de busca, mas de inserção e remoção também –possuem a mesma ordem de complexidade sendo O(log n).

Pode ser concluído, portanto, que a ordem de complexidade dos métodos de inserção, remoção e busca disponíveis na biblioteca para árvores binárias perfeitamente balanceadas e nos métodos padrões do Java são iguais – O(log n).

Ao se tratar do programa, a adaptação do método “quantidadeNos” foi feita porque a estrutura padrão do Java conta com um método próprio (size), cuja ordem de complexidade é O(1), por possuir e atualizar a variável “size” sempre que os métodos de incrementar e remover forem utilizados, da mesma forma em que foi aplicado o método “quantidadeNos” na biblioteca, que também possui complexidade O(1).

Como o desenvolvimento desse app se deu a partir do AppCustomLib, os métodos utilizados foram os aplicados no menu (inserção, remoção, busca, contagem de nós, etc). Entretanto, os métodos de altura e caminhamento em nível não estão presentes pois o TreeSet foca no conjunto (set) e navegabilidade da árvore, visando ordenar e fazer operações de busca, inserção e remoção em tempos logarítmicos, mantendo o estudo dos algoritmos fora do escopo de interesse da estrutura base.

Já a estratégia usada pela estrutura do Java para fazer comparações é aplicada de forma híbrida. O Comparable é priorizado para ordens padronizadas (naturais) ou quando o objeto já o implementa – como as classes Integer e String. Já o Comparator é aplicado para ordens personalizadas e em objetos que não implementam o Comparable.

De forma simples, o TreeSet do Java verifica o uso do Comparator no construtor e, caso não houver, ele tenta converter o objeto para Comparable para fazer as comparações.

6. CONCLUSÃO

O trabalho permitiu validar, de forma teórica e empírica, a influência estrutural sobre o desempenho computacional de árvores binárias de busca. Ao longo das análises, ficou evidente que a escolha da estrutura de dados e, sobretudo, a ordem de entrada dos elementos que a compõem são fatores críticos para a viabilidade de um sistema em escala.

A análise matemática apresentada na Seção 2 estabeleceu as bases formais esperadas: operações de inserção, busca e remoção em uma árvore perfeitamente balanceada possuem complexidade O(log n) em razão da redução pela metade do espaço de busca a cada nível percorrido. Diferente de uma árvore completamente degenerada, onde a estrutura se degenera em uma lista encadeada e a complexidade deteriora para O(n), exigindo, no pior caso, a varredura de todos os elementos.

A operação quantidadeNos() constitui exceção singular a essa dinâmica, mantendo complexidade O(1) independentemente da topologia, graças ao atributo contador totalNos, atualizado incrementalmente a cada modificação da estrutura.

Os experimentos empíricos da Seção 3 reforçam as previsões matemáticas. Ao submeter a Árvore Binária de Busca (BST) aos arquivos de teste, observou-se um crescimento proporcional nos tempos de operação das árvores degeneradas. Ao contrário disso, as árvores balanceadas mantiveram tempos estáveis e inferiores a 0,07 ms independentemente do volume testado, comportamento característico da complexidade logarítmica O(log n). Isso confirma o fato de que, não é o volume de dados em si que degrada o desempenho, mas sim a altura da estrutura resultante após as inserções.

A análise comparativa entre BST e AVL, desenvolvida na Seção 4, aprofundou essa conclusão ao demonstrar a eficácia do mecanismo de auto-balanceamento. Enquanto a BST comum é "refém" da ordem de chegada dos dados, aceitando de forma passiva a degeneração estrutural, a Árvore AVL intervém ativamente a cada operação, aplicando rotações simples ou duplas para manter o fator de equilíbrio de todos os nós. O custo desse mecanismo, observado na fase de inserção de

arquivos balanceados, é justificado pela garantia de performance nas operações subsequentes.

O estudo da biblioteca padrão do Java, conduzido na Seção 5, reforça a relevância prática desses princípios. A classe TreeSet emprega internamente uma estrutura auto-balanceada que assegura complexidade O(log n) para todas as operações fundamentais de inserção, busca e remoção, independentemente do padrão de entrada dos dados. Da mesma forma, o método size() do TreeSet mantém complexidade O(1) por meio de uma variável contadora atualizada incrementalmente.

Em síntese, este trabalho demonstrou que a complexidade algorítmica não é um conceito puramente teórico, mas uma realidade com impacto direto na escalabilidade dos sistemas. Conclui-se que a diferença entre O(n) e O(log n), embora abstrata em pequenos volumes, traduz-se em discrepâncias expressivas sob carga real, tornando o uso de estruturas auto-balanceadas indispensável em aplicações onde a ordem de entrada dos dados não pode ser controlada.